AUDINEXIA

GRC Compliance Automation Platform

Technical Project Report

Author	Lalit Bist
Project	Audinexia — Automated Policy Compliance & GRC Engine
Platform Version	v3.0 (backend engine identifier)
Report Date	July 2026
Repository	E:\backup7 (backend: Flask/Python, frontend: React scaffold)

This report documents the current implementation of Audinexia as built, and separately proposes a future-scope roadmap toward a production-grade GRC (Governance, Risk & Compliance) platform. All figures (control counts, endpoints, frameworks) are drawn directly from the current codebase.

Table of Contents

1. Executive Summary

Audinexia is a policy compliance assessment engine that ingests an organization's written policy documents (privacy policy, security policy, etc.) and evaluates them against structured regulatory and standards frameworks. The system currently supports six frameworks — DPDPA 2023 (India), ISO 27001:2022, GDPR (EU), PCI DSS v4.0, HIPAA, and NIST CSF 2.0 — spanning 55 individually weighted controls. Each control is checked for evidence of required language in the uploaded document, scored, and rolled up into an overall weighted compliance percentage with a Compliant / Partially Compliant / Non-Compliant verdict.

Beyond scoring, the system generates professional HTML/PDF compliance reports, produces a gap-remediation document with suggested replacement language for missing controls, and exposes a live analyst dashboard. The current build is a functional single-service backend (Flask) with no persistence layer, authentication, or automated test suite; these are identified in Section 11 and form the basis of the roadmap in Section 12.

2. Introduction & Problem Statement

Organizations subject to data-protection and security regulations (DPDPA, GDPR, HIPAA, etc.) are required to maintain written policies that demonstrably satisfy specific statutory or standards-body clauses. In practice, policy review against a multi-clause framework is a manual, expert-driven, and time-consuming exercise — typically performed by legal or compliance staff reading a document against a checklist line by line.

2.1 Objective

- Automate first-pass coverage checking of a policy document against a defined set of framework controls.
- Produce a quantified, explainable compliance score (not just pass/fail) with per-control evidence.
- Generate audit-ready reports (HTML and PDF) suitable for sharing with stakeholders or regulators.
- Suggest concrete remediation language for controls found missing, lowering the effort to close gaps.

2.2 Scope

The current scope is document-based static assessment: the system does not verify that an organization's actual technical or operational practices match what its policy states — only that the policy document itself contains language addressing each control's requirements. This distinction is treated as an explicit, honest limitation rather than glossed over (see Section 11).

3. System Architecture

Audinexia is currently a monolithic Flask backend (`backend/app.py`, ~2,360 lines) plus a supporting `core/risk_engine.py` module, with a separate React frontend directory that is presently a project scaffold (component/page folders exist but no application code is implemented yet — the working UI today is the server-rendered `templates/dashboard.html`).

3.1 Request Flow

- **Upload** — client sends a policy file (.txt / .pdf / .docx, up to 50MB) plus a framework selector to `POST /api/scan`.
- **Extraction** — `extract_text()` pulls raw text via direct read (.txt), pdfplumber (.pdf), or python-docx (.docx, including table cells).
- **Control analysis** — for every control in the selected framework, `analyze_control()` checks the extracted text for required phrases (with synonym expansion) and records matched/missing phrases and evidence text.
- **Scoring** — `calculate_weighted_score()` rolls up per-control scores into an overall weighted percentage.
- **Response** — JSON payload of per-control results and counts is returned to the client; reports/PDFs are generated on-demand from separate endpoints using the same result data.

3.2 Module Overview

Component	Responsibility
app.py	Flask app, routes, framework/control definitions, scoring, HTML/PDF report generation
core/risk_engine.py	Maps a control score to a risk level, business impact narrative, attack scenario, and remediation priority
data/dpdpa_controls.json	Standalone JSON control definition for DPDPA (currently unused by the live code path — see Section 11)
templates/dashboard.html	Server-rendered analyst dashboard (single template, ~115KB)
policies/	Sample compliant / partial / non-compliant policy documents used for manual demonstration and validation
frontend/	React project scaffold (components, pages, styles directories) — not yet implemented

4. Technology Stack

Layer	Technology
Backend framework	Flask 2.3.3
Cross-origin support	flask-cors 4.0.0
File handling	werkzeug 2.3.7 (secure_filename, upload handling)
PDF generation	reportlab 4.0.4 (Platypus document templates)
PDF text extraction	pdfplumber (optional dependency, feature-detected at import time)
DOCX text extraction	python-docx (optional dependency, feature-detected at import time)
Frontend (planned)	React (scaffold only — components/pages/styles directories present, unimplemented)

Note: PDF and DOCX support are implemented as soft dependencies — the app checks for `pdfplumber` / `python-docx` at import time and degrades gracefully (returns an explanatory message) if either is not installed, rather than crashing.

5. Compliance Framework Coverage

Six frameworks are implemented, together covering 55 individually weighted controls. Each control carries an ID, clause/section reference, owning role, severity, weight, a list of required phrases (with synonym sets), a plain-language rationale, and example remediation text.

Framework	Controls	Representative Coverage Areas
DPDPA 2023 (India)	12	Consent, notice, data-principal rights, retention, grievance redressal, cross-border transfer
ISO 27001:2022	9	Information security management controls (Annex A-aligned)
GDPR (EU)	8	Lawful basis, data subject rights, breach notification, DPO, international transfers
PCI DSS v4.0	6	Cardholder data protection, access control, encryption, monitoring
HIPAA	6	Protected health information safeguards, breach notification, access controls
NIST CSF 2.0	14	Identify / Protect / Detect / Respond / Recover function controls

Each control definition also includes a `remediation_example` *string used both in scan results and in the auto-generated revised-policy document (Section 9).*

6. Core Features

6.1 Multi-format Document Scanning

Accepts .txt, .pdf, and .docx policy documents up to 50MB. Text is extracted uniformly regardless of source format, including table-cell content in DOCX files, before being run through the control-matching engine.

6.2 Synonym-aware Phrase Matching

Rather than requiring an exact phrase match, each required control phrase (e.g. “explicit consent”) maps to an expanded synonym list (e.g. “informed consent”, “opt-in”, “clear affirmative action”) so that reasonable paraphrasing in a real policy is still recognized as coverage.

6.3 Weighted Compliance Scoring

Each control has an importance weight (reflecting severity/penalty exposure). The overall score is a weight-adjusted average of per-control coverage, not a simple unweighted pass count, so critical controls (e.g. consent, security safeguards) influence the final verdict more than minor ones.

6.4 Evidence Extraction

For every matched phrase, the engine extracts the surrounding sentence from the source document as human-readable evidence, with separator lines and boilerplate stripped, so a reviewer can see exactly why a control was marked compliant.

6.5 Risk Contextualization

The risk engine (Section 7) converts each control's numeric score into a risk tier with an associated business-impact statement, illustrative attack scenario, and remediation priority/timeframe.

6.6 Professional Report Export

Results can be exported as a formatted HTML report or a multi-section PDF (cover page, score summary, per-control breakdown, evidence, and recommendations) via ReportLab.

6.7 AI-Assisted Gap Remediation

The revise-policy endpoint identifies every control with missing phrases and compiles a remediation document with the specific suggested replacement/addition language for each gap, optionally rendered as a downloadable PDF.

6.8 Live Analyst Dashboard

A server-rendered dashboard view (`/dashboard`) presents scan results in an interactive, styled interface for walkthroughs and demonstrations.

7. Scoring & Risk Methodology

7.1 Per-control Scoring

For a control with *N* required phrases, the engine counts how many are found (directly or via synonym) in the document text. The raw control score is:

$$\texttt{raw\_score} = (\texttt{phrases\_found} / \texttt{total\_required\_phrases}) \times 100$$

Raw Score	Status	Risk Level
≥ 80%	Compliant	Low
50–79%	Partially Compliant	Medium
< 50%	Non-Compliant	High

7.2 Overall (Weighted) Score

The overall document score is the weight-normalized average of all per-control raw scores, not a simple mean:

$$\texttt{overall\_score} = \Sigma(\texttt{weight}_i \times \texttt{score}_i / 100) / \Sigma(\texttt{weight}_i) \times 100$$

7.3 Risk Engine

`core/risk_engine.py` maps a control's score to one of four risk tiers, each with a color code, remediation timeframe, and qualitative business-impact statement:

Risk Tier	Score Range	Remediation Window	Business Impact
Critical	< 30	0–7 days	Severe — potential data breach / maximum regulatory exposure
High	30–49	7–30 days	Significant — compliance gaps, potential data exposure
Medium	50–69	30–60 days	Moderate — partial compliance gaps
Low	≥ 70	60–90 days	Minimal — minor improvements needed

8. API Reference

Endpoint	Purpose
GET /	Service metadata: status, supported frameworks, allowed formats, max file size
POST /api/scan	Uploads a document + framework selector; returns per-control results and overall score
POST /api/export-report	Generates a downloadable HTML compliance report from prior scan results
POST /api/export-pdf	Generates a downloadable PDF compliance report from prior scan results
POST /api/revise-policy	Analyzes a document, identifies missing controls, and returns (or renders as PDF) suggested remediation text
GET /dashboard	Server-rendered analyst dashboard view

All endpoints respond with CORS headers permitting cross-origin requests, and binary (PDF/HTML) responses explicitly expose `Content-Disposition` *so browser-based clients can trigger file downloads correctly.*

9. Report Generation & AI-Assisted Remediation

Two distinct output types are generated from a completed scan:

- **Compliance Report** (HTML or PDF) — a formatted document with cover section, overall score, per-control status table, matched evidence, and framework-specific business-impact language.
- **Revised-Policy Remediation Document** — rather than only reporting what is missing, the `/api/revise-policy` endpoint compiles the specific suggested replacement language (from each control's `remediation_example`) for every gap found, so a policy author has concrete next-step text rather than just a deficiency list.

This directly reduces the manual effort GRC/legal teams normally spend translating an audit finding into actual policy language.

10. Validation & Test Scenarios

The repository ships four reference policy documents under `policies/`, used for manual functional validation of the scoring engine across the compliance spectrum:

Sample Document	Framework	Expected Score	Expected Status
Fully_Compliant_Policy.txt	DPDPA 2023	85–100%	Compliant
ISO27001_Compliant_Policy.txt	ISO 27001:2022	85–100%	Compliant
Partially_Compliant_Policy.txt	DPDPA 2023	40–70%	Partially Compliant
Non_Compliant_Policy.txt	DPDPA 2023	0–30%	Non-Compliant

This is currently a manual validation process (documented in `policies/README.txt`) run by uploading each sample through the dashboard and confirming the score falls in the expected band. No automated unit or integration test suite exists yet for the scoring logic — see Section 11.

11. Current Limitations

In keeping with an evidence-based, no-overclaiming approach, the following gaps are acknowledged explicitly rather than left implicit:

- **No persistence layer** — scans are stateless; results exist only in the HTTP response and are not stored, so there is no historical record, audit trail, or ability to track remediation over time.
- **No authentication or multi-tenancy** — the API and dashboard have no login, roles, or per-organization data isolation.
- **Document-only assessment** — the engine verifies that a policy *states* a control is in place, not that the control is technically or operationally implemented.
- **Keyword/synonym matching, not semantic understanding** — coverage detection is phrase-based; unusual phrasing outside the synonym lists can under-score a genuinely compliant policy.
- **No automated test suite** — validation is manual (Section 10); scoring-logic regressions would not be caught automatically.
- **Monolithic single-file backend** — `app.py` (~2,360 lines) mixes routing, scoring, and report-generation code, which limits maintainability at larger scale.
- **Unused duplicate control data** — `data/dpdpa_controls.json` defines a second, slightly different DPDPA control set that is not referenced by the live code path (the authoritative definitions live inline in `app.py`).
- **Frontend not yet implemented** — the React scaffold under `frontend/` contains no application code; the working UI is the server-rendered dashboard template.

12. Future Scope

The following enhancements are proposed to evolve Audinexia from a document-scoring tool into a production-oriented GRC system of record. None of these are implemented in the current build; they are prioritized roughly by the value they add relative to the existing architecture.

12.1 Persistence & Audit Trail

Introduce a relational store (organizations, assessments, control_results) so every scan is retained with timestamp and assessor, enabling historical trend reporting and satisfying the basic auditability expectation of any real compliance tool.

12.2 Control Crosswalk

Restructure framework-specific controls to reference a shared, framework-agnostic control library (e.g. one “encryption in transit/at rest” control mapped to both DPDPA-5 and ISO 27001 A.8.24), so a single scan can project estimated coverage against multiple frameworks at once — a capability the current one-framework-per-scan design does not offer.

12.3 Evidence & Remediation Workflow

Add a human-in-the-loop review layer: reviewer confirm/override of auto-detected scores, attached evidence files per control, and remediation tracking (owner, due date, status). This converts the tool from a one-shot scanner into an ongoing system of record, which is the defining characteristic of GRC software versus a compliance linter.

12.4 Risk Register & Maturity Scoring

Track identified risks as persistent, ownable items (not just point-in-time scan output), and report compliance maturity on a staged model (e.g. Initial / Managed / Defined / Quantitatively Managed / Optimizing) alongside the raw percentage score.

12.5 Authentication, RBAC & Multi-tenancy

Add organization-scoped accounts and role-based access (admin / auditor / viewer), required given the sensitivity of compliance assessment data.

12.6 Third-Party / Vendor Risk Assessment

Reuse the existing scoring engine to assess vendor-supplied policies and roll results into a vendor risk register — a natural extension of the current architecture with high real-world relevance (vendor questionnaires are a core GRC use case).

12.7 Continuous Compliance Monitoring

Move beyond one-shot scans toward scheduled re-assessment and drift detection (e.g. flagging when a previously compliant policy has not been re-verified within a defined interval, or when a framework's underlying requirements change).

12.8 Engineering Housekeeping

- Split the monolithic app.py into routing / scoring / report-generation modules.
- Add an automated test suite covering the scoring engine against the existing sample policy documents.

- Remove or reconcile the unused data/dpdpa_controls.json duplicate control set.
- Initialize version control (the project currently has no git history).
- Build out the React frontend scaffold, or formally retire it in favor of the server-rendered dashboard.

13. Conclusion

Audinexia demonstrates a working end-to-end pipeline for automated policy compliance assessment: multi-format document ingestion, weighted multi-framework scoring across 55 controls in six regulatory and standards frameworks, risk contextualization, and professional report generation with AI-assisted remediation suggestions. As a static-analysis, single-service tool it is well-suited as a first-pass compliance screening aid and as a demonstration of applied GRC domain knowledge.

To operate as production GRC software, the system needs the additions outlined in Section 12 — most importantly persistence/audit trail, a human review workflow, and access control — since those are what distinguish a system of record from a scoring utility. The roadmap in this report is structured so each addition builds directly on the existing scoring engine rather than requiring a rewrite.

End of report. Generated from the current state of the codebase at E:\backup7.